Network Intrusion Detection System - Detailed Project Report

Network Intrusion Detection System Using Machine Learning

Detailed Project Report

Abstract

This project develops a Machine Learning-based Network Intrusion Detection System (NIDS) for classifying network traffic flows into normal and malicious categories. The system is built around the CICIDS2017 dataset, a widely used benchmark dataset for intrusion detection research. The project implements a full training and evaluation pipeline, including dataset loading, label grouping, data cleaning, feature scaling, class imbalance handling, model training, model comparison, model persistence, and offline PCAP-based inference.

The main task is multi-class classification. The final label space includes `Normal`, `DoS`, `DDoS`, `PortScan`, `BruteForce`, `Bot`, and `Infiltration`. Four machine learning models are evaluated: Logistic Regression, K-Nearest Neighbors, Random Forest, and XGBoost. Because CICIDS2017 is highly imbalanced, especially for minority classes such as `Bot` and `Infiltration`, the project also evaluates model behavior before and after conservative imbalance handling. The imbalance strategy combines Random Under-Sampling for the majority `Normal` class with light SMOTE for selected minority classes.

Experimental results show that Random Forest without imbalance handling achieves the best full-dataset Test Macro-F1 score of `0.9378`, while XGBoost with conservative imbalance handling shows the clearest improvement on minority-class detection, increasing Test Macro-F1 from `0.8243` to `0.9215` and improving `Infiltration` F1 from `0.00` to `0.71`. Logistic Regression performs poorly on rare attack classes, while KNN achieves moderate full-dataset performance but is computationally expensive during inference.

The project also supports offline PCAP testing. PCAP files are converted into CICIDS-like flow features using a patched CICFlowMeter-based extractor or a Scapy fallback. The trained model then predicts traffic classes from the extracted flows. PCAP testing demonstrates that the XGBoost model with imbalance handling can detect DoS-like behavior in the `task3.dos_victim.pcap` file, while other models tend to classify the tested PCAP flows as `Normal`. This highlights both the usefulness and the limitations of applying CICIDS-trained models to external PCAP traffic.

1. Introduction

1.1 Background

Network Intrusion Detection Systems are used to monitor network traffic and detect suspicious or malicious activity. Traditional intrusion detection systems often rely on signature-based rules. Although signature-based methods can detect known attacks reliably, they usually struggle with unseen attack patterns, modified payloads, and zero-day behavior.

Machine learning offers a different approach. Instead of checking only fixed rules, a model learns statistical patterns from historical network traffic. Once trained, it can classify new traffic based on flow-level behavior such as packet counts, byte counts, packet lengths, inter-arrival times, protocol flags, and traffic direction statistics.

This project focuses on flow-based intrusion detection. A flow represents a communication session or a group of packets sharing common properties such as source, destination, ports, and protocol. Flow-based features are more compact than raw packets and are commonly used in intrusion detection datasets such as CICIDS2017.

1.2 Project Motivation

The goal of this project is not only to train a high-accuracy model, but also to understand model behavior under dataset imbalance and evaluate whether trained models can be used on offline PCAP traffic.

The project investigates several practical questions:

- Can machine learning classify common network attacks from CICIDS2017 flow features?
- How does class imbalance affect the result?
- Does imbalance handling improve all models equally?
- Which model is most suitable for reporting and which model is most useful for PCAP inference?
- Can a model trained on CICIDS2017 generalize to external PCAP files?

1.3 Project Scope

The project includes:

- CICIDS2017 CSV dataset loading.
- Multi-class label grouping.
- Data cleaning and preprocessing.
- Train/validation/test splitting.
- Conservative dataset imbalance handling.
- Training and comparing four models:
- Logistic Regression
- K-Nearest Neighbors
- Random Forest

- XGBoost
- Saving trained artifacts.
- Evaluating models with classification reports, Macro-F1, Weighted-F1, and cross-validation where applicable.
- Offline PCAP inference through flow extraction.

The project no longer includes rule-based log analysis. The previous `log_analyze.py` workflow was removed to keep the project focused on machine learning and PCAP-based NIDS inference.

---

2. Project Structure

The project is organized into a small number of main directories:

```
NIDS---Network-Intrusion-Detection-System/
├── data/
│   ├── raw/CICIDS2017/
│   └── pcap/
├── models/final/
├── experiments/models/
├── outputs/
├── reports/training_runs/
├── src/
│   ├── data/
│   ├── models/
│   ├── pipeline/
│   └── utils/
├── live_analyze.py
├── main.py
├── requirements.txt
├── README.md
└── REPORT.md
```

2.1 `data/`

The `data/` directory stores input data.

```
data/raw/CICIDS2017/
```

This folder contains the CICIDS2017 CSV flow files used for training and evaluation.

```
data/pcap/
```

This folder contains offline PCAP files used for inference testing.

2.2 `models/final/`

This is the default model artifact directory used by PCAP inference.

It contains:

```
model.pkl
scaler.pkl
label_encoder.pkl
features.pkl
```

These files are required for inference:

- `model.pkl`: trained classifier.
- `scaler.pkl`: fitted `StandardScaler`.
- `label_encoder.pkl`: fitted `LabelEncoder`.
- `features.pkl`: ordered feature list used during training.

2.3 `experiments/models/`

This folder stores model artifacts from different experiment runs. It is separated from `models/final/` to keep the main model directory clean.

Examples:

```
experiments/models/report_full_xgb_with_imbalance/
experiments/models/report_full_rf_no_imbalance/
experiments/models/report_full_knn_no_imbalance/
experiments/models/report_full_lr_no_imbalance/
```

Each experiment folder contains the same artifact structure:

```
model.pkl
scaler.pkl
label_encoder.pkl
features.pkl
```

2.4 `src/`

The `src/` directory contains the reusable source code.

```
src/data/
```

Contains data loading and preprocessing logic.

```
src/models/
```

Contains model definitions and prediction helpers.

```
src/pipeline/
```

Contains the end-to-end training pipeline.

```
src/utils/
```

Contains shared configuration values.

2.5 `live_analyze.py`

This file handles PCAP inference. It extracts flow features from a PCAP file, aligns them with the trained model's feature list, scales the features, runs prediction, and prints a NIDS-style report.

2.6 `main.py`

This is the command-line entry point. It supports two main modes:

```
--train
--pcap
```

---

3. Dataset

3.1 Dataset Source

The project uses the CICIDS2017 flow dataset. The dataset is stored in:

```
data/raw/CICIDS2017/
```

The loaded dataset contains:

```
2,830,743 rows
79 columns including Label
```

3.2 CSV Files Used

The full dataset is loaded from the following CICIDS2017 CSV files:

```
Monday-WorkingHours.pcap_ISCX.csv
Tuesday-WorkingHours.pcap_ISCX.csv
Wednesday-workingHours.pcap_ISCX.csv
Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv
Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv
Friday-WorkingHours-Morning.pcap_ISCX.csv
Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv
Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
```

3.3 Feature Type

The dataset uses flow-level features rather than raw packets. Examples include:

- Flow duration.
- Total forward packets.

- Total backward packets.
- Total length of forward packets.
- Total length of backward packets.
- Packet length mean, min, max, and standard deviation.
- Flow bytes per second.
- Flow packets per second.
- Flow inter-arrival time statistics.
- Forward and backward inter-arrival time statistics.
- TCP flag counts.
- Header length statistics.
- Active and idle time statistics.

3.4 Original Class Distribution

After label grouping, the full dataset has the following class distribution:

Class	Samples
Normal	2,273,097
DoS	252,661
PortScan	158,930
DDoS	128,027
BruteForce	13,835
Bot	1,966
Infiltration	47

3.5 Dataset Imbalance Problem

The dataset is highly imbalanced. `Normal` traffic dominates the dataset, while `Infiltration` has only 47 samples. This creates several issues:

- Accuracy can become misleadingly high.
- Weighted-F1 can remain high even when rare classes are not detected.
- A model may learn to predict majority classes well while ignoring minority attacks.
- Rare-class metrics can fluctuate heavily because their test support is very small.

For this reason, Macro-F1 is emphasized in the report. Macro-F1 gives equal importance to each class and is more informative for imbalanced multi-class classification.

---

4. Label Grouping

The raw CICIDS2017 labels are mapped into broader attack categories.

Original Label	Grouped Label
`BENIGN`, `Benign`, `Normal`	`Normal`
`DoS Hulk`, `DoS GoldenEye`, `DoS Slowloris`, `DoS Slowhttptest`, `DoS`	`DoS`
`DDoS`	`DDoS`
`PortScan`	`PortScan`
`FTP-Patator`, `SSH-Patator`, `BruteForce`	`BruteForce`
`Bot`	`Bot`
`Web Attack Brute Force`, `Web Attack XSS`, `Web Attack Sql Injection`	`Web`
`Infiltration`, `Heartbleed`	`Infiltration`

In the final full-dataset experiments, the observed classes are:

```
Normal
DoS
DDoS
PortScan
BruteForce
Bot
Infiltration
```

---

5. Preprocessing Pipeline

The preprocessing logic is implemented in:

```
src/data/preprocess.py
```

5.1 Column Name Normalization

Column names are stripped to remove leading or trailing spaces:

```
df_clean.columns = df_clean.columns.str.strip()
```

This is necessary because some CICIDS2017 files contain column names with inconsistent spacing.

5.2 Metadata Removal

Metadata columns are removed before training:

```
Flow ID
Source IP
Source Port
Destination IP
Destination Port
Protocol
Timestamp
```

Equivalent fields from PCAP flow extractors are also removed:

```
flow_id
src_ip
src_port
dst_ip
dst_port
protocol
timestamp
```

These fields are removed because they identify traffic endpoints rather than general behavior. Keeping them could cause the model to learn dataset-specific identifiers instead of robust intrusion patterns.

5.3 Missing and Infinite Values

The preprocessing stage handles invalid numerical values:

```
df_clean.replace([np.inf, -np.inf], np.nan, inplace=True)
df_clean.fillna(0, inplace=True)
```

This ensures that the model does not receive infinite or missing values.

5.4 Numeric Conversion

All feature columns are converted to numeric values:

```
df_clean[col] = pd.to_numeric(df_clean[col], errors="coerce")
```

Any failed conversions become `NaN` and are later filled with `0`.

5.5 Train/Validation/Test Split

The cleaned dataset is split into:

Split	Ratio
Train	70%
Validation	10%
Test	20%

The split is stratified when possible so that class proportions are preserved.

5.6 Label Encoding

Text labels are transformed into numerical labels using `LabelEncoder`.

The saved encoder is later used to convert numerical predictions back into class names during inference.

5.7 Feature Scaling

The project uses `StandardScaler`.

Important detail:

- The scaler is fitted only on the training set.
- Validation and test sets use `transform`.
- PCAP inference uses the saved `scaler.pkl`.

This prevents data leakage from validation/test data into the training process.

---

6. Class Imbalance Handling

The imbalance handling function is:

```
handle_imbalance(X_train, y_train)
```

It uses a conservative strategy:

1. Random under-sampling for the majority `Normal` class.
2. Light SMOTE for selected minority classes.
3. No aggressive oversampling for extremely rare classes.

6.1 Random Under-Sampling

The `Normal` class is reduced to at most 300,000 training samples:

```
target_normal = min(300000, normal_count)
```

This reduces majority-class dominance while still keeping a large number of real normal samples.

6.2 Conservative SMOTE

SMOTE is applied only when a class has enough samples to support synthetic generation.

Condition	Strategy
Class has fewer than 100 samples	Do not increase
`Bot`	Increase up to 1.5x, capped at 2,500
Class has fewer than 2,000 samples	Increase up to 1.5x, capped at 4,000
Class has fewer than 10,000 samples	Increase up to 1.25x, capped at 12,000
Large classes	Keep unchanged

This approach avoids making the test results look artificially good by generating too many synthetic samples from very small classes.

6.3 Training Distribution After Imbalance Handling

For the full dataset, the training set after imbalance handling becomes:

Class	Training Samples After Handling
Normal	300,000
DoS	176,863
PortScan	111,251
DDoS	89,618
BruteForce	12,000
Bot	2,064
Infiltration	33

6.4 Important Observation

Imbalance handling does not improve every model. It improves XGBoost clearly, but it slightly reduces the overall Macro-F1 for Random Forest and KNN. This is expected because resampling changes the density and distribution of the training data. Distance-based methods like KNN are especially sensitive to this.

---

7. Models Used

The project evaluates four models:

```
Logistic Regression
K-Nearest Neighbors
Random Forest
XGBoost
```

7.1 Logistic Regression

Logistic Regression is used as a linear baseline.

Current configuration:

```
solver = saga
max_iter = 300
tol = 1e-3
n_jobs = -1
random_state = 42
```

Role in project:

- Provides a simple baseline.
- Useful for comparison against non-linear models.
- Helps show that the problem is not easily solved by a linear classifier.

Observed behavior:

- It performs poorly on rare attack classes.
- It does not detect `Bot`, `BruteForce`, or `Infiltration` in the full-dataset test set.
- It produces convergence warnings on the full dataset.

7.2 K-Nearest Neighbors

KNN is used as a distance-based baseline.

Current configuration:

```
n_neighbors = 5
n_jobs = -1
```

Role in project:

- Compares distance-based classification against tree-based models.
- Helps evaluate whether local feature-space similarity is sufficient for NIDS classification.

Observed behavior:

- It achieves moderate Macro-F1 on the full dataset.
- It is slow during inference because prediction requires distance comparison against a large training set.
- It performs worse after imbalance handling because under-sampling changes sample density.

7.3 Random Forest

Random Forest is a tree-based ensemble model.

Current configuration:

```
n_estimators = 100
n_jobs = -1
random_state = 42
```

Role in project:

- Main high-performing baseline.
- Strong on CICIDS2017 internal train/test split.
- Robust to many feature interactions.

Observed behavior:

- It achieves the best full-dataset Test Macro-F1.
- It performs slightly better without imbalance handling than with imbalance handling.
- It does not generalize well to the tested PCAP files in model-only inference; it predicts all tested PCAP flows as `Normal`.

7.4 XGBoost

XGBoost is the main boosted tree model.

Current configuration:

```
n_estimators = 600
max_depth = 4
learning_rate = 0.04
subsample = 0.8
colsample_bytree = 0.75
reg_alpha = 2.0
reg_lambda = 8.0
gamma = 2.0
min_child_weight = 10
max_delta_step = 1
objective = multi:softmax
eval_metric = merror
tree_method = hist
n_jobs = -1
random_state = 42
```

Role in project:

- Main model for imbalance-handling analysis.
- Provides feature importance output.
- Performs best in PCAP model-only testing among the four models.

Observed behavior:

- XGBoost improves strongly after imbalance handling.
- It detects `DoS` flows in `task3.dos_victim.pcap`.
- It is more sensitive to minority/attack-like flow patterns than RF/KNN/LR in the tested PCAP files.

7.5 XGBoost Sample Weights

When XGBoost is trained with imbalance handling, the pipeline also uses clipped balanced sample weights:

```
min weight = 0.75
max weight = 2.00
```

This prevents the model from overreacting to extremely rare classes.

---

8. Training Pipeline

The training pipeline is implemented in:

```
src/pipeline/train_pipeline.py
```

8.1 Training Flow

The full training process is:

```
Load CICIDS2017 CSV files
        |
Clean data and map labels
        |
Split into train/validation/test
        |
Apply imbalance handling if enabled
        |
Encode labels
        |
Scale features
        |
Train selected model
        |
Evaluate validation set
        |
Evaluate test set
        |
Run sampled cross-validation if possible
        |
Save model artifacts
```

8.2 Training Command

Train XGBoost with imbalance handling:

```
python3 main.py --train data/raw/CICIDS2017 --model xgb
```

Train Random Forest:

```
python3 main.py --train data/raw/CICIDS2017 --model rf
```

Train Logistic Regression:

```
python3 main.py --train data/raw/CICIDS2017 --model lr
```

Train KNN:

```
python3 main.py --train data/raw/CICIDS2017 --model knn
```

8.3 Training Without Imbalance Handling

Use:

```
python3 main.py --train data/raw/CICIDS2017 --model xgb --no-imbalance
```

8.4 Training With a Custom Output Directory

Use:

```
python3 main.py --train data/raw/CICIDS2017 --model xgb --model-dir models/final
```

8.5 Training on a Sample

Use:

```
python3 main.py --train data/raw/CICIDS2017 --model xgb --sample 100000
```

This is useful for quick experiments, but final metrics should be based on the full dataset.

8.6 Saved Artifacts

Each training run saves:

```
model.pkl
<model_name>.pkl
scaler.pkl
label_encoder.pkl
features.pkl
```

These artifacts are required for inference.

---

9. Evaluation Metrics

The project uses:

- Precision.
- Recall.
- F1-score.
- Macro-F1.
- Weighted-F1.
- Cross-validation Weighted-F1 where possible.

9.1 Accuracy

Accuracy measures the total percentage of correct predictions.

However, because CICIDS2017 is heavily imbalanced, accuracy can be misleading. A model can achieve very high accuracy by predicting majority classes well while failing minority attacks.

9.2 Precision

Precision measures how many predicted samples of a class are actually correct.

For intrusion detection, low precision means more false alarms.

9.3 Recall

Recall measures how many true samples of a class are detected.

For intrusion detection, low recall means the system misses attacks.

9.4 F1-score

F1-score balances precision and recall.

9.5 Macro-F1

Macro-F1 averages F1-score across classes equally.

This is the most important metric in this project because it reveals whether minority classes are being detected.

9.6 Weighted-F1

Weighted-F1 averages F1-score by class support.

Because `Normal` has very high support, Weighted-F1 can remain high even if rare classes perform poorly.

---

10. Full Dataset Results

10.1 Overall Results

Model	Imbalance Handling	Validation Macro-F1	Validation Weighted-F1	Test Macro-F1	Test Weighted-F1	CV Weighted-F1
Logistic Regression	No	0.4788	0.9224	0.4791	0.9224	Skipped
Logistic Regression	Yes	0.5030	0.8940	0.4554	0.8938	0.8821 +/- 0.0018
KNN	No	0.8818	0.9981	0.8769	0.9982	0.9834 +/- 0.0011
KNN	Yes	0.8252	0.9916	0.8378	0.9914	Skipped
XGBoost	No	0.8682	0.9987	0.8243	0.9988	Skipped
XGBoost	Yes	0.9459	0.9984	0.9215	0.9985	0.9981 +/- 0.0002
Random Forest	No	0.9536	0.9988	0.9378	0.9987	Skipped
Random Forest	Yes	0.9497	0.9983	0.9280	0.9985	0.9979 +/- 0.0002

10.2 Per-Class Test F1

Model	Imbalance	Bot	BruteForce	DDoS	DoS	Infiltration	Normal	PortScan
Logistic Regression	No	0.00	0.00	0.75	0.78	0.00	0.96	0.87
Logistic Regression	Yes	0.00	0.00	0.69	0.76	0.00	0.93	0.80
KNN	No	0.70	0.99	1.00	1.00	0.46	1.00	1.00
KNN	Yes	0.55	0.91	1.00	0.99	0.46	0.99	0.96
XGBoost	No	0.78	1.00	1.00	1.00	0.00	1.00	1.00
XGBoost	Yes	0.75	1.00	1.00	1.00	0.71	1.00	1.00
Random Forest	No	0.77	1.00	1.00	1.00	0.80	1.00	1.00
Random Forest	Yes	0.79	1.00	1.00	1.00	0.71	1.00	1.00

10.3 Result Interpretation

Random Forest without imbalance handling achieves the best full-dataset Test Macro-F1:

```
Test Macro-F1 = 0.9378
Test Weighted-F1 = 0.9987
```

XGBoost benefits the most from imbalance handling:

```
XGBoost no imbalance Test Macro-F1 = 0.8243
XGBoost with imbalance Test Macro-F1 = 0.9215
```

The most important improvement is for `Infiltration`:

```
Infiltration F1 before imbalance = 0.00
Infiltration F1 after imbalance  = 0.71
```

KNN performs worse after imbalance handling:

```
KNN no imbalance Test Macro-F1 = 0.8769
KNN with imbalance Test Macro-F1 = 0.8378
```

This is because KNN depends heavily on sample density. Under-sampling changes the structure of the feature space and can reduce performance.

Logistic Regression is the weakest model:

```
LR no imbalance Test Macro-F1 = 0.4791
LR with imbalance Test Macro-F1 = 0.4554
```

It fails to detect several minority attack classes.

---

11. Model Selection

11.1 Best Model by Internal CICIDS2017 Metrics

The best full-dataset Macro-F1 is achieved by:

```
Random Forest without imbalance handling
```

It is the strongest model on the internal train/test split.

11.2 Best Model for Imbalance-Handling Story

The best model for demonstrating the effect of imbalance handling is:

```
XGBoost with conservative imbalance handling
```

This model shows clear improvement after imbalance handling and improves rare-class detection.

11.3 Best Model for PCAP Testing

Based on tested PCAP files, the most useful model is:

```
XGBoost with conservative imbalance handling
```

It is the only tested model that detected DoS flows in `task3.dos_victim.pcap`.

11.4 Recommended Final Choice

For the project report, the recommended conclusion is:

```
Random Forest achieved the best overall Macro-F1 on the CICIDS2017 test split, while
XGBoost with conservative imbalance handling showed the clearest improvement on
minority attack classes and performed best in PCAP-based model-only inference.
```

---

12. PCAP Inference

12.1 Purpose

PCAP inference is used to test the trained model on offline network capture files.

The goal is to simulate how the NIDS might behave on traffic outside the original CICIDS2017 CSV dataset.

12.2 PCAP Inference Flow

The PCAP inference flow is:

```
Input PCAP
   |
Extract flow features
   |
Normalize feature columns
   |
Clean data
   |
Align columns with training feature list
   |
Scale using saved scaler
   |
Predict using saved model
   |
Decode labels
   |
Print NIDS report
```

12.3 Flow Extraction

The project uses:

```
cicflowmeter-patched
```

as the primary extractor.

If extraction fails or returns no flows, the project can use:

```
scapy-fallback
```

The fallback extractor is less complete but helps the project continue when CICFlowMeter cannot produce usable flows.

12.4 Default PCAP Model Directory

By default, PCAP testing uses:

```
models/final/
```

Required files:

```
models/final/model.pkl
models/final/scaler.pkl
models/final/label_encoder.pkl
models/final/features.pkl
```

12.5 PCAP Test Command

Default model:

```
python3 main.py --pcap data/pcap/test1.pcap
```

Specific model:

```
python3 main.py --pcap data/pcap/task3.dos_victim.pcap --model-dir
experiments/models/report_full_xgb_with_imbalance
```

Random Forest example:

```
python3 main.py --pcap data/pcap/task3.dos_victim.pcap --model-dir
experiments/models/report_full_rf_no_imbalance
```

KNN example:

```
python3 main.py --pcap data/pcap/task3.dos_victim.pcap --model-dir
experiments/models/report_full_knn_no_imbalance
```

Logistic Regression example:

```
python3 main.py --pcap data/pcap/task3.dos_victim.pcap --model-dir
experiments/models/report_full_lr_no_imbalance
```

12.6 PCAP Test Results

The project tested multiple PCAP files using all four models.

`task3.dos_victim.pcap`

Extracted flows:

```
41,830 flows
extractor: cicflowmeter-patched
```

Results:

Model	Normal	DoS	Interpretation
Logistic Regression	41,830	0	Predicted all flows as Normal
KNN	41,830	0	Predicted all flows as Normal
Random Forest	41,830	0	Predicted all flows as Normal
XGBoost with imbalance	36,368	5,462	Detected DoS-like flows

`task1.dos_victim.pcap`

Extracted flows:

```
224 flows
extractor: cicflowmeter-patched
```

Results:

Model	Prediction
Logistic Regression	224 Normal
KNN	224 Normal
Random Forest	224 Normal
XGBoost	224 Normal

`task3.dos_attacker.pcap`

Extracted flows:

```
43,525 flows
extractor: cicflowmeter-patched
```

Results:

Model	Prediction
Logistic Regression	43,525 Normal
KNN	43,525 Normal
Random Forest	43,525 Normal
XGBoost	43,525 Normal

`task1.dos_attacker.pcap`

Extracted flows:

```
5,114 flows
extractor: scapy-fallback
```

Results:

Model	Prediction
Logistic Regression	5,114 Normal
KNN	5,114 Normal
Random Forest	5,114 Normal
XGBoost	5,114 Normal

12.7 PCAP Testing Interpretation

The PCAP results show that internal CICIDS2017 metrics do not guarantee strong generalization to external PCAP traffic.

Important observations:

- XGBoost with imbalance handling is the only tested model that detected DoS on `task3.dos_victim.pcap`.
- RF performs best on the internal CICIDS2017 split but predicts all tested PCAP flows as `Normal`.
- LR and KNN also predict all tested PCAP flows as `Normal`.
- PCAP inference is affected by domain shift between CICIDS2017 training data and the extracted PCAP flows.
- PCAP inference is also affected by feature extraction quality.

The PCAP results should be described as model predictions, not ground truth labels.

---

13. How to Use the Project

13.1 Install Dependencies

```
pip install -r requirements.txt
```

13.2 Train a Model

Train XGBoost:

```
python3 main.py --train data/raw/CICIDS2017 --model xgb --model-dir models/final
```

Train Random Forest:

```
python3 main.py --train data/raw/CICIDS2017 --model rf --model-dir
experiments/models/rf_test
```

Train without imbalance handling:

```
python3 main.py --train data/raw/CICIDS2017 --model rf --no-imbalance --model-dir
experiments/models/rf_no_imbalance
```

Train on a sample:

```
python3 main.py --train data/raw/CICIDS2017 --model xgb --sample 100000 --model-dir
experiments/models/xgb_sample100k
```

13.3 Test a PCAP File

Using the default final model:

```
python3 main.py --pcap data/pcap/test1.pcap
```

Using a specific model:

```
python3 main.py --pcap data/pcap/task3.dos_victim.pcap --model-dir
experiments/models/report_full_xgb_with_imbalance
```

13.4 Enable Supplemental PCAP Signatures

The project contains optional PCAP signature logic in `live_analyze.py`. By default, model-only inference is used. To enable supplemental signature alerts:

```
python3 main.py --pcap data/pcap/task3.dos_victim.pcap --signatures
```

In the main project analysis, model-only results are emphasized to avoid mixing machine learning predictions with manually defined rules.

---

14. Saved Logs and Outputs

Training logs are stored in:

```
reports/training_runs/
```

Important summary file:

```
reports/training_runs/model_results_summary.md
```

PCAP test outputs are stored in:

```
outputs/
```

Examples:

```
outputs/pcap_task3_dos_victim_all_models.txt
outputs/pcap_task1_dos_victim_all_models.txt
outputs/pcap_task3_dos_attacker_all_models.txt
outputs/pcap_task1_dos_attacker_all_models.txt
```

---

15. Limitations

15.1 Dataset Imbalance

Some classes are extremely rare. `Infiltration` has only 47 samples in the full dataset. This makes reliable learning and evaluation difficult.

15.2 Small Test Support for Rare Classes

`Infiltration` has only 9 samples in the test set. A small number of correct or incorrect predictions can change its F1-score significantly.

15.3 Domain Shift in PCAP Testing

The model is trained on CICIDS2017 flow CSV data. External PCAP files may have different feature distributions. This can cause the model to classify attacks as `Normal`.

15.4 Flow Extraction Differences

PCAP features extracted by CICFlowMeter or Scapy fallback may not perfectly match the original CICIDS2017 feature generation process.

15.5 KNN Inference Cost

KNN is slow during prediction because it compares new samples with stored training samples.

15.6 Logistic Regression Convergence

Logistic Regression does not converge well on the full dataset and performs poorly on minority classes.

---

16. Future Work

Possible improvements:

1. Add a dedicated CSV-flow prediction command.
2. Improve PCAP feature extraction to better match CICIDS2017.
3. Add probability thresholding for XGBoost predictions.
4. Add SHAP-based model explainability.
5. Separate binary detection from multi-class classification.
6. Try LightGBM or CatBoost.
7. Evaluate on more external PCAP datasets.
8. Add a clearer experiment tracking system.
9. Add confusion matrix plots and feature importance visualizations to the report.
10. Investigate why RF performs well on CICIDS2017 but poorly on tested PCAPs.

---

17. Conclusion

This project successfully implements a full Machine Learning-based NIDS pipeline using CICIDS2017. It supports data loading, cleaning, label grouping, imbalance handling, model training, model evaluation, artifact saving, and offline PCAP inference.

The experiments show that Random Forest achieves the best internal CICIDS2017 full-dataset Macro-F1 score. However, XGBoost with conservative imbalance handling provides the strongest evidence that imbalance handling improves rare attack detection, especially for `Infiltration`. In PCAP inference, XGBoost is also the only tested model that detects DoS-like traffic in `task3.dos_victim.pcap`.

The project demonstrates an important real-world lesson: high test performance on a benchmark dataset does not automatically guarantee strong performance on external PCAP traffic. Domain shift and flow extraction differences matter. Therefore, the project should present RF as the strongest internal benchmark model and XGBoost with imbalance handling as the most practical model for minority-class and PCAP-oriented analysis.